

```
In [ ]: # Initialize Otter
import otter
grader = otter.Notebook("practice02.ipynb")
```

Student names and e-mails:

YOUR NAME — your@calvin.edu

YOUR NAME — your@calvin.edu

## Practice 02 — Cleaning, Grouping, and Visualizing Data

In this practice you will work with a real dataset about footballers from the English Premier League. Each task is tagged with the SLO it covers:

| SLO | Description                                                                                               |
|-----|-----------------------------------------------------------------------------------------------------------|
| 03A | Clean and transform text data using string operations in DataFrames                                       |
| 03B | Group data to calculate aggregates such as counts, means, or sums                                         |
| 03C | Produce and interpret histograms, scatter plots, line plots, and bar charts to explore a dataset visually |

### The Dataset: Premier League 2023/24 Player Stats



This dataset contains season-long statistics for individual players from the 2023/24 Premier League season — goals, assists, progressive passing and carrying, expected goals, and more. It is a simplified version of a dataset originally published on [Kaggle \(https://www.kaggle.com/datasets/orkunaktas/premier-league-all-players-stats-2324\)](https://www.kaggle.com/datasets/orkunaktas/premier-league-all-players-stats-2324). Each row is one player.

## Column descriptions (the ones you'll use most):

| Column      | Meaning                                                                                                    |
|-------------|------------------------------------------------------------------------------------------------------------|
| Player      | The player's name                                                                                          |
| Nation      | A lowercase 2-letter code and an uppercase 3-letter code together, e.g. "eng ENG"                          |
| Pos         | The player's position(s), e.g. "FW" or "FW,MF" for a player who plays both                                 |
| Age         | The player's age during the season                                                                         |
| Gls         | Total goals scored                                                                                         |
| Ast         | Total assists                                                                                              |
| xG          | Expected goals — an estimate of how many goals the player <i>should</i> have scored, based on shot quality |
| CrdY , CrdR | Total yellow / red cards received                                                                          |
| PrgC , PrgP | Progressive carries / passes — forward-moving ball actions                                                 |
| Team        | The player's Premier League club                                                                           |

`Nation` and `Pos` are exactly the kind of columns Monday's class warned you about: readable to a human, not yet usable by `groupby()`.

```
In [ ]: import pandas as pd
import plotly.express as px
```

The cell below loads the data from a CSV file into a pandas DataFrame called `players`. Run it and look at the first few rows.

```
In [ ]: players = pd.read_csv('https://cs.calvin.edu/courses/data/202/fsantos/premier-league.csv')
players.head()
```

Notice two things before we do anything else:

- `Nation` packs *two* codes into one string, e.g. "eng ENG" — not usable for grouping by country yet.
- `Pos` can hold *more than one* position per player, e.g. "FW,MF" — not usable for grouping by position yet either.

Part 1 fixes both.

## Part 1 — Cleaning String Data (SLO 03A)

### Task 03A.1 — Extracting the Nation Code (1 pt)

The `Nation` column looks like "eng ENG" — a lowercase code, a space, then the uppercase 3-letter code you actually want.

Using `.str.split(' ')` and indexing (or another string method of your choice), create a new column `players['Nation_Code']` containing just the uppercase 3-letter code, e.g. "ESP" , "ENG" .

```
In [ ]: players['Nation_Code'] = players['Nation'].str.split(' ').str[1] # SOLUTION
        players[['Player', 'Nation', 'Nation_Code']].head()
```

```
In [ ]: grader.check("03A.1")
```

### Task 03A.2 — Flagging Multi-Position Players (2 pts)

The `Pos` column holds one or more positions per player, e.g. "FW,MF" .

1. Split `Pos` on the comma into a new column `players['Pos_List']` , where each value is a *list* of positions (`.str.split(',')`).
2. From `Pos_List` , create a boolean column `players['Is_Multi_Position']` that is `True` for players with more than one position.
3. From `Pos_List` , create `players['Primary_Pos']` holding just the *first* listed position for each player (their main position).
4. Assign the total number of multi-position players to `n_multi_position` .

*Hint: `.str.len()` on a column of lists gives you each list's length; `.str[0]` gives you its first element.*

```
In [ ]: players['Pos_List'] = players['Pos'].str.split(',') # SOLUTION
        players['Is_Multi_Position'] = players['Pos_List'].str.len() > 1 # SOLUTION
        players['Primary_Pos'] = players['Pos_List'].str[0] # SOLUTION
        n_multi_position = players['Is_Multi_Position'].sum() # SOLUTION
        print(f'Multi-position players: {n_multi_position}')
        players[['Player', 'Pos', 'Pos_List', 'Is_Multi_Position', 'Primary_Pos']].head()
```

```
In [ ]: grader.check("03A.2")
```

## Task 03A.3 — Mapping Codes to Full Country Names (2 pts)

Even after extracting `Nation_Code`, you still only have an abbreviation — turning `"ESP"` into `"Spain"` isn't something regex or case rules can do, because it's not a spelling problem, it's an *abbreviation* problem. That's exactly the distinction Monday's class made with `"SF" → "San Francisco"`: it needed an explicit lookup dictionary, not a pattern.

Here's a starter dictionary covering six countries:

```
nation_map = {
    'ENG': 'England', 'ESP': 'Spain', 'BRA': 'Brazil',
    'ARG': 'Argentina', 'FRA': 'France', 'POR': 'Portugal',
}
```

Use `.replace()` with `nation_map` to create `players['Nation_Name']` from `players['Nation_Code']`. (Codes *not* in the dictionary will simply stay as their 3-letter code — that's expected; a real lookup table would need many more entries.)

```
In [ ]: nation_map = {
        'ENG': 'England', 'ESP': 'Spain', 'BRA': 'Brazil',
        'ARG': 'Argentina', 'FRA': 'France', 'POR': 'Portugal',
        }

players['Nation_Name'] = players['Nation_Code'].replace(nation_map) # SOLUTION
players[['Player', 'Nation_Code', 'Nation_Name']].drop_duplicates('Nation_Code').head(10)
```

```
In [ ]: grader.check("03A.3")
```

## Part 2 — Grouping and Aggregating (SLO 03B)

Use **named aggregation** where you can — it's the syntax we've been recommending, and it's what the tests below expect your column names to match:

```
new_dataframe = (
    dataframe
    .groupby('Column', as_index=False)
    .agg(new_column1=('Col', 'function'),
        new_column2=('Col', 'function'))
)
```

### Task 03B.1 — Total Goals by Nation (2 pts)

Group `players` by `Nation_Code` and use named aggregation to compute `total_goals` (the sum of `GLs`) per nation. Sort so the highest-scoring nations come first, and keep only the top 5. Assign the result to `top5_nations`.

```
In [ ]: top5_nations = (
    players
    .groupby('Nation_Code', as_index=False)
    .agg(total_goals=('Gls', 'sum'))
    .sort_values('total_goals', ascending=False)
    .head(5)
) # SOLUTION
top5_nations
```

```
In [ ]: grader.check("03B.1")
```

We'll now create age groups ( < 25 , 25-30 , > 30 ) using `pd.cut()` . Run the cell below — you don't need to modify it.

```
In [ ]: bins = [0, 25, 30, 100] # Defining bins for age groups
labels = ['< 25', '25-30', '> 30'] # Defining labels for the bins
players['Age Group'] = pd.cut(players['Age'], bins=bins, labels=labels, right=False)
players[['Player', 'Age', 'Age Group']].head()
```

## Task 03B.2 — Cards by Age Group (2 pts)

Group `players` by `Age Group` and use named aggregation to compute both `total_yellow` (sum of `CrdY`) and `total_red` (sum of `CrdR`) per group, in a single `.agg(...)` call. Assign the result to `cards_by_age` .

*Hint: pass `observed=True` to `.groupby()` to avoid an extra empty-category warning.*

```
In [ ]: cards_by_age = (
    players
    .groupby('Age Group', as_index=False, observed=True)
    .agg(total_yellow=('CrdY', 'sum'), total_red=('CrdR', 'sum'))
) # SOLUTION
cards_by_age
```

```
In [ ]: grader.check("03B.2")
```

## Task 03B.3 — Age Range by Team (2 pts)

Not every summary has a ready-made function. Using a **custom lambda** (like Monday's `x.max() - x.min()` example), group `players` by `Team` and compute the *range* of `Age` on each squad — the oldest player's age minus the youngest's. Sort from largest range to smallest. Assign the result to `age_range_by_team` .

```
In [ ]: age_range_by_team = (
    players
    .groupby('Team')['Age']
    .agg(lambda x: x.max() - x.min())
    .sort_values(ascending=False)
) # SOLUTION
age_range_by_team.head()
```

```
In [ ]: grader.check("03B.3")
```

## Part 3 — Visual Encodings (SLO 03C)

| Question                                 | Plot             |
|------------------------------------------|------------------|
| What's the distribution of one number?   | <b>Histogram</b> |
| How do two numbers relate to each other? | <b>Scatter</b>   |
| How do groups compare?                   | <b>Bar</b>       |

```
In [ ]: fig_example = px.bar(
        top5_nations,
        x='Nation_Code',
        y='total_goals',
        title='Top 5 Nations by Total Goals (Premier League 2023/24)',
        labels={'Nation_Code': 'Nation', 'total_goals': 'Total Goals'})
fig_example.show()
```

### Task 03C.1 — Distribution of Goals (2 pts)

Create a **histogram** of `Gls` (goals scored) across all players.

- x-axis: `'Gls'`
- A descriptive title
- Axis label via the `labels=` argument

Assign the figure to `fig1` and display it.

```
In [ ]: # BEGIN SOLUTION
fig1 = px.histogram(
    players,
    x='Gls',
    nbins=20,
    title='Distribution of Goals Scored (Premier League 2023/24)',
    labels={'Gls': 'Goals Scored'})
# END SOLUTION
fig1.show()
```

## Task 03C.2 — Expected vs. Actual Goals (3 pts)

Using your `Primary_Pos` column from Part 1, create a **scatter plot** comparing each player's `xG` (expected goals) to their actual `Gls` (goals scored).

Requirements:

- Chart type: **scatter**
- x-axis: `'xG'`
- y-axis: `'Gls'`
- **Color** encoding: `'Primary_Pos'`
- A title and axis labels

Assign to `fig2` and display it.

```
In [ ]: # BEGIN SOLUTION
fig2 = px.scatter(
    players,
    x='xG',
    y='Gls',
    color='Primary_Pos',
    title='Expected Goals vs. Actual Goals by Position (2023/24)',
    labels={'xG': 'Expected Goals (xG)', 'Gls': 'Actual Goals', 'Primary_Pos': 'Position'})
# END SOLUTION
fig2.show()
```

## Task 03C.3 — Evaluate Your Visualization (2 pts)

Look critically at `fig2`. In **3–5 sentences**, answer:

1. Does coloring by `Primary_Pos` help you spot which positions tend to over- or under-perform their `xG`? Why or why not?
2. What would you change to make the pattern clearer?
3. What does this plot **not** show — what information is hidden or lost?

*Edit the cell below and write your answer.*

*Your answer here.*



# Just for fun: what is the point of sports analytics?

(Not graded — just something to chew on.)

- Check out this [article \(https://theconvivialsociety.substack.com/p/the-limits-of-optimization\)](https://theconvivialsociety.substack.com/p/the-limits-of-optimization) by L. M. Sacasas commenting on how sports today have changed with the introduction of quantification and analytics.
- Are we really having fun with that?

"So you can't blame anyone for the way the game has developed," Jacobs concludes. "It has become more rational, with a better command of the laws of probability, and stricter, more rigorous canons of efficiency."

"It's worth pausing to consider wherein the purported rationality lies. It is the logic of competition. Within the sporting world, of course, the point is to win, and to do so in a way that can be clearly determined quantitatively. There are no grounds for anyone to ask a manager or a player to pursue a strategy that will diminish their competitive edge. Most of life, however, is not a game with quantifiable outcomes, and probably shouldn't be treated as such. However, the triumph of technique in Ellul's sense encourages the competitive mode of experience. Indeed, quantification itself invites it. This dynamic can be put to beneficial use, and, in clearly delineated circumstances, is perfectly appropriate. But applied uncritically and indiscriminately or even nefariously (see e.g. social media metrics) it can introduce destructive tendencies and eclipse qualitative or otherwise unquantifiable values. Generally speaking, quantification and the logic of optimization which it encourages tend to transform our field of experience into points of aggression, as the sociologist Hartmut Rosa has aptly put it. Data-driven optimization is, in this sense, a way of perceiving the world. And what may matter most about this is not necessarily what it allows us to see, but it keeps us from perceiving: in short, all that cannot be quantified or measured."

## Submission

Make sure you have run all cells in your notebook in order before running the cell below, so that all images/graphs appear in the output. The cell below will generate a zip file for you to submit. **Please save before exporting!**

Submit the .zip file to our Moodle assignment page.

```
In [ ]: # Save your notebook first, then run this cell to export your submission.  
grader.export(run_tests=True)
```